

# { AISD

DOŁNE GRANICE

Dane:

$[a_i]_{i=1}^n \rightarrow$  elementy

WYNIK:

TAK - jeżeli istnieje  $a_i, a_j$ , gdy  $i \neq j$ , ale  $a_i = a_j$   
(czyli istnieją różne elementy)

NIE - wpp (elementy są różne)

IN Dostępny algorytm rozwiązujący powyższy problem wykonuje  $\Omega(n \log n)$  porównań (algorytm oparty na porównaniach).

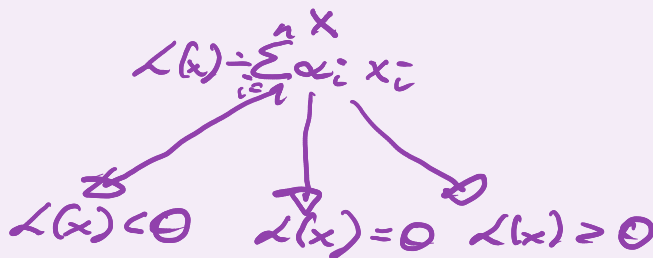
d-d:

Ułożymy  $[a_i]_{i=1}^n$  a punktem w przestrzeni

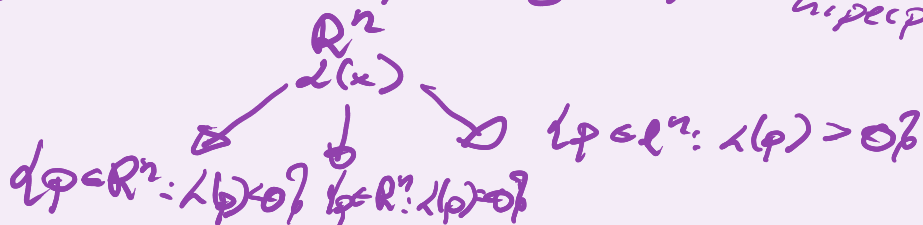
$\mathbb{R}^n$ , gdzie  $i$ -ta współrzędna  $x$  to  $i$ -ty wyraz ciągu  $(a_i)$ .

Wtedy zamiast pytać się o różnicę elementów, pytamy się o różnicę współrzędnych.

Algorytm zadaje pytania o kombinacje liniowe punktu i porównuje je do 0. Dla danej kombinacji liniowej  $L(x)$  mamy trzy odpowiedzi ( $L(x) > 0$ ,  $L(x) = 0$ ,  $L(x) < 0$ ). Ułożymy dzieło algorytmu z dwiema ternarymi:

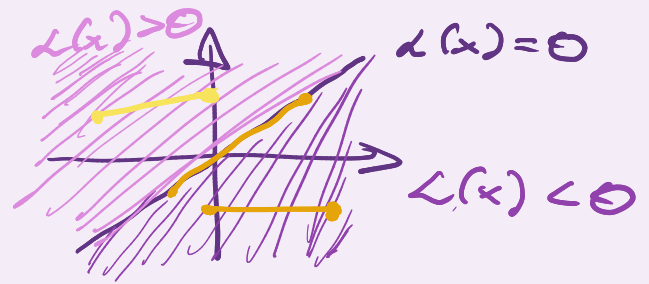
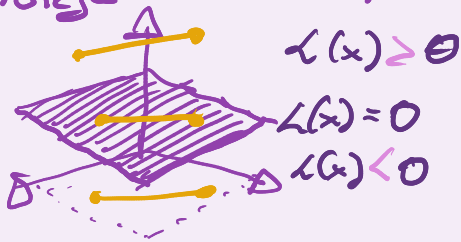


Każde pytanie rozkłada nam  $\mathbb{R}^n$  na podzbiory  $\mathbb{R}^n$  podzielone hiperpłaszczyzną, a te dzielone innymi hiperpłaszczyznami.



Każdy z takich zbiorów jest dystryktem (przyda się formalne wyjaśnienie - to do) to znaczy ma odcińki między 2 punktami  $\forall$  punkty należące do tego zbioru.

Intuicja dla  $\mathbb{R}^3, \mathbb{R}^2$ :



LIŚCIEM DRZEWIA nazywamy moment w którym algorytm odpowiada TAK albo NIE (tj. Jeżeli punkt tam trafi to oznacza, że ma różne znieś, gdy odpowie NIE albo TAK wpp). Czyli jest to miejsce, gdzie algorytm NIE, co ma odpowiedzieć

Rozważmy punkty  $P_1, P_2, \dots, P_n$  będące permutacjami  $n$ -elementowymi. Pokażemy, że każdy należy do różnych liści co oznaczać będzie, że tych liści jest  $\geq n!$

Złożymy przeciwie, niech  $P_j, P_i$  należy do tego samego liścia. Z racji, że PERMUTACJAMI, mają różne elementy, więc algorytm odpowie NIE. W szczególności a definicji liścia taka odpowiedź będzie dla wszystkich punktów z niego. Niech  $k$  będzie najmniejsze, t.j.:

$$(P_i(k_i) = k \wedge P_j(k_i) \neq k) \wedge (P_j(k_j) = k \wedge P_i(k_j) \neq k)$$

Gdzie  $P_i(m)$  to  $m$ -ty element  $P_i(m)$

To znaczy  $k$  jest najmniejszym elementem (NIE INDEKSEM!) takim, że jest w obu permutacjach  $P_i, P_j$  na różnych pozycjach. Takie  $k$  istnieje, bo  $P_i \neq P_j$ .

Przykład:

$P_i = 52314$  W tym przykładzie najmniejszym  
 $P_j = 42351$  takim  $k$  jest  $1$ , a  $k_i = 4$ , a  $k_j = 5$

Definiujemy: pozycja występowania  $k$  w  $P_i, P_j$ .  $P_i(k_i) = k$

$$f(x) = x_{k_i} - x_{k_j} \rightarrow \text{pozycja } k \text{ w } P_j \text{ j. } P_j(k_j) = k$$

dla powyższego przykładu

$$f(P_i) = P_i(k_i) - P_i(k_j) = 4 - 1 = 3$$

$$f(P_j) = P_j(k_i) - P_j(k_j) = 5 - 1 = 4$$

Bez straty ogólności założymy, że  $k_i < k_j$   
(gdyby  $k_i = k_j$  to  $k$  byłoby na tej samej pozycji,  
a to byłoby sprzeczność z jego definicją), jeśli  
nie to zamienimy permutacje nawzajem.

Wówczas  $P_i(k_i) \neq P_j(k_i)$  (bo  $P_i(k_i) = k$ , a  $P_j(k_i)$  to  
coś innego).

$P_i(k_j) \neq P_j(k_j)$  (bo  $P_j(k_j) = k$ , ale  $P_i(k_j) \neq k$

bo w permutacji nie ma kopii elementu, a  
pamiętajmy, że w  $P_i$   $k$  znajduje się na pozycji  $k_i$  istotnie  
różnej niż  $k_j$ ).

Zauważmy, że  $P_j(k_i) > k$ , dlatego, że  $k$  jest  
najmniejszym elementem, że  $k$  występuje w obu  
permutacjach na różnych pozycjach. Analogicznie  
 $P_i(k_j) > k$ .

Wtedy

$$\cdot f(P_i) = P_i(k_i) - P_i(k_j) = k - P_i(k_j) < 0 \quad (\text{bo } P_i(k_j) > k)$$

$$\cdot f(P_j) = P_j(k_i) - P_j(k_j) = P_j(k_i) - P_j(k_j) > 0 \quad (\text{bo } P_j(k_i) > k)$$

Zauważmy, że  $f$  jest ciągła. W definicji ciągłości mieliśmy, że dla każdego  $\epsilon > 0$  możemy znaleźć tak „blisko” położone sobie argumenty  $x^{(1)}, x^{(2)}$ , że  $|f(x_1) - f(x_2)| < \epsilon$ .

Dla dowodu moglibyśmy np. wziąć takie  $x^{(1)}, x^{(2)}$ , że

$$\begin{aligned} \cdot x_{k_i}^{(1)} &= x_{k_i}^{(2)} \\ \cdot x_{k_j}^{(2)} &= x_{k_j}^{(1)} - \epsilon', \text{ t.j. } 0 < \epsilon' < \epsilon \end{aligned}$$

Wtedy  $|f(x^{(1)}) - f(x^{(2)})|$  wygląda następująco:

$$|x_{k_i}^{(1)} - x_{k_j}^{(1)} - x_{k_i}^{(2)} + x_{k_j}^{(2)}| = |x_{k_i}^{(1)} - x_{k_j}^{(1)} - x_{k_j}^{(1)} + x_{k_j}^{(1)} + \epsilon'| = |\epsilon'| < \epsilon$$

Przypomnijmy, że rozważony liść jest zbiorem wypukłym.

Zatem punkty na odcinku łączącym  $P_i$  oraz  $P_j$  należą do tego samego liścia.

Z własności funkcji ciągłej mamy, że na tym odcinku  $f$  przyjmie wszystkie wartości z przedziału  $[f(P_i), f(P_j)]$ , a że  $f(P_i) < 0$  i  $f(P_j) > 0$ , to  $\exists \hat{x}$  znajdujący się na tym odcinku, że:

$$f(\hat{x}) = 0.$$



Ale z definicji  $f$  mamy  $f(\hat{x}) = \hat{x}_{k_i} - \hat{x}_{k_j}$ . Zatem  $\hat{x}_{k_i} - \hat{x}_{k_j} = 0 \iff \hat{x}_{k_i} = \hat{x}_{k_j}$ .

Ale  $k_i < k_j$ , więc w szczególności  $k_i \neq k_j$ . Zatem  $\hat{x}$  posiada współrzędne o tej samej wartości na różnych miejscach. Czyli algorytm powinien odpowiedzieć TAK do tego punktu. Ale odpowiedź dla punktów z jednego liścia (z definicji)

jest wspólna, a dla  $P_i$  oraz  $P_j$  była odpowiedź przecząc (NIE), zatem algorytm dał złą odpowiedź dla  $x^1$  i mamy sprzeczność. Zatem  $P_i$  oraz  $P_j$  należą do różnych liści. Permutacji jest  $n!$  zatem liczba liści  $\geq n!$ .

Algorytm skończy się wraz z dotarciem do 1 z liści, czyli w najgorszym przypadku przejdzie najdłuższą drogę do liścia (czyli wykona  $h$ -kroków, gdzie  $h$  - to wysokość drzewa). Jeśli algorytm był optymalny, to drzewo będzie zbalansowane, czyli liczba liści będzie wynosić  $3^h$  (bo drzewo jest ternarne), czyli

$$3^h \geq n! \quad | \log_3$$
$$h \geq \log_3 n!$$

Na WD1 pokazaliśmy, że  $\Theta(n \log n!)$  to  $\Theta(n \log n)$  zatem tyle będzie około wynosić wysokość (z dokładnością do stałej, które dostaniemy w wyniku zmiany podstawy logarytmu).

Zatem takie jest też dolne ograniczenie.